

PRACTICAL FILE

C++ SEM-2

Niket Aggarwal

25739

B.Sc.(Hons) CS

Common Array function

In all program these are already imported:

```
#include <iostream>
#include <cmath>
#include <fstream>
#include <string>
using namespace std;
```

To take input in array:

```
void inp(int arr[], int n){
    int j = 1;
    for (int i = 0; i < n; i++){
        cout << "Element " << j << ":";
        cin >> arr[i];
        j++;
    }
    return;
}
```

To display elements of array:

```
void display(int a[], int x){
    cout << "Array:";
    for ( int i = 0; i < x; i++)
    {
        cout << a[i] << " ";
    }
    cout << endl;
    return;
}
```

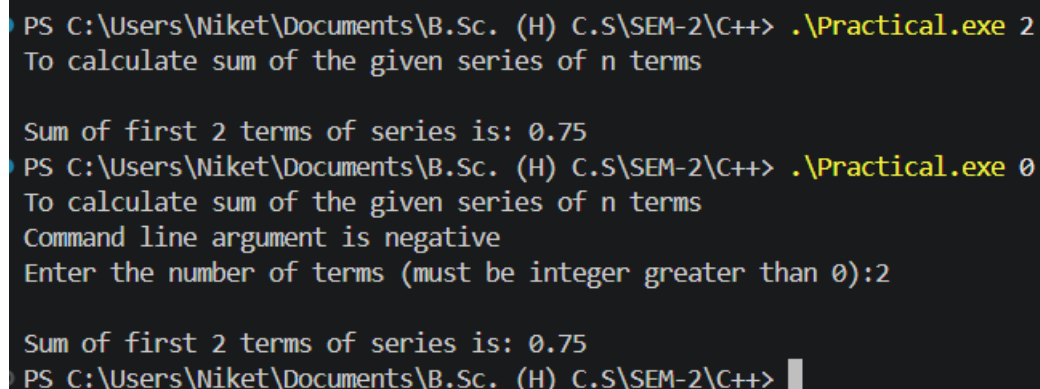
Q1- Write a program to compute the sum of the first n terms of the following series:
$$=1-1/2^2+1/3^3-\dots 1/n^n$$

The number of terms n is to be taken from the user through the command line. If the command line argument is not found then prompt the user to enter the value of n.

Program:

```
int main(int argc, char *a[]){
    cout << "To calculate sum of the given series of n terms" << endl;
    int n;
    if (argc > 1){
        n = stoi(a[1]);
        if (n <= 0){
            cout << "Command line argument is negative" << endl;
            do{
                cout << "Enter the number of terms (must be integer
greater than 0):";
                cin >> n;
            } while (n <= 0);
        }
    }else{
        do{
            cout << "Enter the number of terms (must be integer greater
than 0):";
            cin >> n;
        } while (n <= 0);
    }
    cout << endl;
    double sum = 0, term = 0;
    int m = 1;
    for (int i = 1; i <= n; i++){
        term = (1.0 / pow(i, i)) * m;
        m *= -1;
        sum += term;
    }
    cout << "Sum of first " << n << " terms of series is: " << sum;
    return 0;
}
```

Output:



```
PS C:\Users\Niket\Documents\B.Sc. (H) C.S\SEM-2\C++> .\Practical.exe 2
To calculate sum of the given series of n terms

Sum of first 2 terms of series is: 0.75
PS C:\Users\Niket\Documents\B.Sc. (H) C.S\SEM-2\C++> .\Practical.exe 0
To calculate sum of the given series of n terms
Command line argument is negative
Enter the number of terms (must be integer greater than 0):2

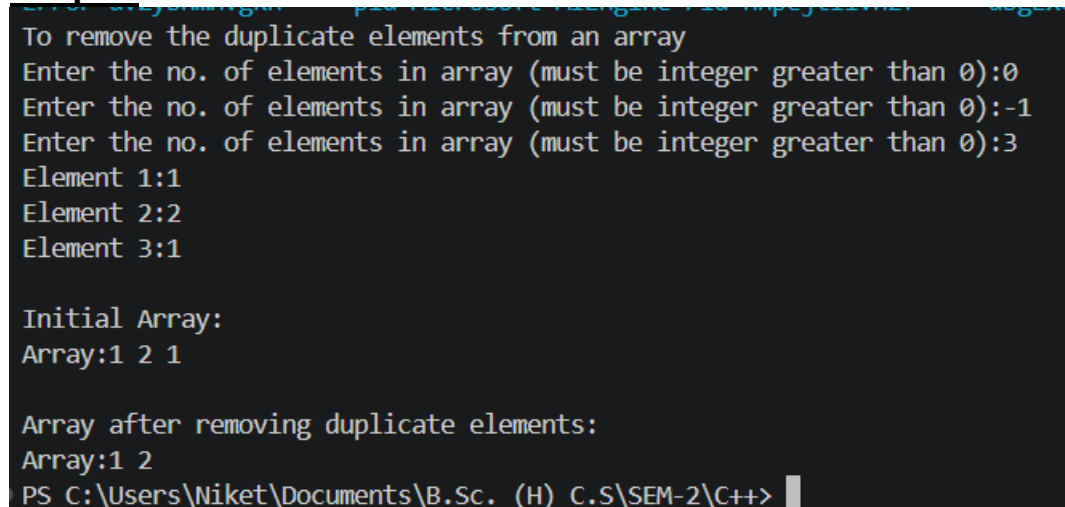
Sum of first 2 terms of series is: 0.75
PS C:\Users\Niket\Documents\B.Sc. (H) C.S\SEM-2\C++> |
```

Q2- Write a program to remove the duplicates from an array.

Program:

```
int main(){
    cout << "To remove the duplicate elements from an array" << endl;
    int n;
    do{
        cout << "Enter the no. of elements in array (must be integer greater than 0):";
        cin >> n;
    } while (n <= 0);
    int arr[n];
    inp(arr, n);
    cout << endl;
    cout << "Initial Array:" << endl;
    display(arr, n);
    int a;
    for (int i = 0; i < n; i++){
        for (int j = i + 1; j < n; j++){
            if (arr[j] == arr[i]){
                for (int k = j; k < n - 1; k++){
                    arr[k] = arr[k + 1];
                    n--;
                    j--;
                }
            }
        }
    }
    cout << endl;
    cout << "Array after removing duplicate elements:" << endl;
    display(arr, n);
    return 0;
}}
```

Output:



```
To remove the duplicate elements from an array
Enter the no. of elements in array (must be integer greater than 0):0
Enter the no. of elements in array (must be integer greater than 0):-1
Enter the no. of elements in array (must be integer greater than 0):3
Element 1:1
Element 2:2
Element 3:1

Initial Array:
Array:1 2 1

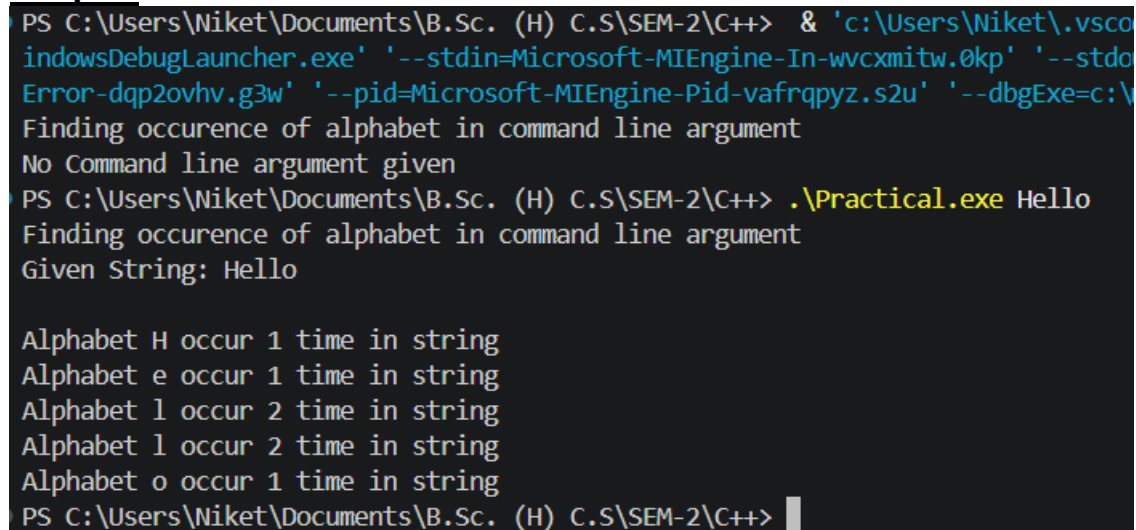
Array after removing duplicate elements:
Array:1 2
PS C:\Users\Niket\Documents\B.Sc. (H) C.S\SEM-2\C++>
```

Q3- Write a program that prints a table indicating the number of occurrences of each alphabet in the text entered as command line arguments.

Program:

```
int main(int argc, char* argv[]){
    cout << "Finding occurrence of alphabet in command line argument"<<endl;
    char* str;
    if (argc == 1){
        cout << "No Command line argument given"<<endl;
        return 1;
    }else {
        str = argv[1];
    }
    cout << "Given String: " << argv[1] << endl << endl;
    int c;
    for (int i = 0; str[i]!='\0'; i++){
        c=0;
        for(int j = 0; str[j]!='\0'; j++){
            if (str[i] == str[j]){
                c++;
            }
        }
        cout << "Alphabet " << str[i] <<" occur " << c << " time in string" <<
endl;
    }
    return 0;
}
```

Output:



```
PS C:\Users\Niket\Documents\B.Sc. (H) C.S\SEM-2\C++> & 'c:\Users\Niket\.vscode\WindowsDebugLauncher.exe' '--stdin=Microsoft-MIEngine-In-wvcxmitw.0kp' '--stdout=Error-dqp2ovhv.g3w' '--pid=Microsoft-MIEngine-Pid-vafrqpyz.s2u' '--dbgExe=c:\Windows\System32\cmd.exe'
Finding occurrence of alphabet in command line argument
No Command line argument given
PS C:\Users\Niket\Documents\B.Sc. (H) C.S\SEM-2\C++> .\Practical.exe Hello
Finding occurrence of alphabet in command line argument
Given String: Hello

Alphabet H occur 1 time in string
Alphabet e occur 1 time in string
Alphabet l occur 2 time in string
Alphabet l occur 2 time in string
Alphabet o occur 1 time in string
PS C:\Users\Niket\Documents\B.Sc. (H) C.S\SEM-2\C++>
```

Q4- Write a menu driven program to perform string manipulation (without using inbuilt string functions):

- a. Show address of each character in string
- b. Concatenate two strings.
- c. Compare two strings
- d. Calculate length of the string (use pointers)
- e. Convert all lowercase characters to uppercase
- f. Reverse the string
- g. Insert a string in another string at a user specified position

Program:

```
void address(char str[]){
    cout << "For String :" << str;
    for (int i = 0; str[i]!='\0'; i++){
        cout << "Address of the character " << str[i] << " is:" << (void*)&str[i]
<< endl;
    }
    return;
}

void concat(char str1[], char str2[]){
    char str3[50];
    int i;
    for (i = 0; str1[i]!='\0'; i++){
        str3[i] = str1[i];
    }
    for (int j = 0; str2[j]!='\0'; j++){
        str3[i] = str2[j];
        i++;
    }
    str3[i]='\0';
    cout << "New string:" << str3;
    return;
}

void compare(char str1[], char str2[]) {
    int i = 0;
    while (str1[i] != '\0' && str2[i] != '\0') {
        if (str1[i] < str2[i]) {
            cout << str1 << " comes first in dictionary";
            return;
        }else if(str1[i] > str2[i]){
            cout << str2 << " comes first in dictionary";
            return;
        }
        i++;
    }
    cout << "Bothe are equal";
    return;
}

int length(char str[]){
    char *pr = str;
    int c=0;
    while (*pr!='\0'){
        c++;
        pr++;
    }
}
```

```

        return c;
    }
    void upper(char str[]){
        char res[20];
        int j;
        for (j = 0; str[j]!='\0'; j++){
            if (str[j] >= 'a' && str[j] <= 'z'){
                res[j]=str[j]-32;
            }else{
                res[j]=str[j];
            }
        }
        res[j]='\0';
        cout << str << " after Uppercase " << res;
        return;
    }
    void reverse(char str[]){
        char res[20];
        int i=0;
        for (int j = (length(str)-1);j != -1; j--){
            res[i]=str[j];
            i++;
        }
        res[i]='\0';
        cout << str << " after Reversing " << res;
        return ;
    }
    void insert(char str1[], char str2[], int x){
        char str3[50];
        int i=0;
        for (int k = 0; str1[k] != '\0'; k++){
            if (k == x){
                for (int j = 0; str2[j] != '\0'; j++){
                    str3[i++] = str2[j];
                }
            }
            str3[i++] = str1[k];
        }
        str3[i]='\0';
        cout << "New string:" << str3;
        return;
    }
    int main(){
        cout << "Enter string for string operations" << endl;
        char str1[20];
        cout << "Enter a string1: ";
        cin >> str1;
        char str2[20];
        cout << "Enter a string2: ";
        cin >> str2;
        cout << endl;
        while (true){
            cout << "\tMenu for string operation" << endl;
            cout << "1.Show address of each character in string\n2.Concatenate two strings\n3.Compare two strings\n4.Calculate length of the string (use pointers)\n5.Convert all lowercase characters to uppercase\n6.Reverse the string\n7.Insert a string in another string at a user specified position\n8.Exit";
            int x;
            cout << endl;

```

```

cout << "Enter your choice:";
cin >> x;
switch (x){
    case 1:
        address(str1);
        cout << endl;
        address(str2);
        break;
    case 2:
        concat(str1,str2);
        break;
    case 3:
        compare(str1,str2);
        break;
    case 4:
        cout << "Length of " << str1 << " is:" << length(str1);
        cout << endl;
        cout << "Length of " << str2 << " is:" << length(str2);
        break;
    case 5:
        upper(str1);
        cout << endl;
        upper(str2);
        break;
    case 6:
        reverse(str1);
        cout << endl;
        reverse(str2);
        break;
    case 7:
        int y;
        cout << "Enter the index:";
        cin >> y;
        insert(str1,str2,y);
        break;
    case 8:
        cout << "..EXIT..";
        break;
    default:
        cout << "Enter Accordinly";
        break;
}
if (x==8){
    break;
}
cout << endl << endl;
}
return 0;
}

```

Output:

```
Error-luc1jvkc.zzm' '--pid=Microsoft-MIEngine-Pid-in5kqwuh.hn0' '--
Enter string for string operations
Enter a string1: Hello
Enter a string2: WORLD

      Menu for string operation
1.Show address of each character in string
2.Concatenate two strings
3.Compare two strings
4.Calculate length of the string (use pointers)
5.Convert all lowercase characters to uppercase
6.Reverse the string
7.Insert a string in another string at a user specified position
8.Exit
Enter your choice:1
For String :HelloAddress of the character H is:0x5ffe50
Address of the character e is:0x5ffe51
Address of the character l is:0x5ffe52
Address of the character l is:0x5ffe53
Address of the character o is:0x5ffe54

For String :WORLDAddress of the character W is:0x5ffe30
Address of the character O is:0x5ffe31
Address of the character R is:0x5ffe32
Address of the character L is:0x5ffe33
Address of the character d is:0x5ffe34

      Menu for string operation
1.Show address of each character in string
2.Concatenate two strings
3.Compare two strings
4.Calculate length of the string (use pointers)
5.Convert all lowercase characters to uppercase
6.Reverse the string
```

```
5.Convert all lowercase characters to uppercase
6.Reverse the string
7.Insert a string in another string at a user specified position
8.Exit
Enter your choice:2
New string:HelloWORLD
```

Menu for string operation

```
1.Show address of each character in string
2.Concatenate two strings
3.Compare two strings
4.Calculate length of the string (use pointers)
5.Convert all lowercase characters to uppercase
6.Reverse the string
7.Insert a string in another string at a user specified position
8.Exit
Enter your choice:3
Hello comes first in dictionary
```

Menu for string operation

```
1.Show address of each character in string
2.Concatenate two strings
3.Compare two strings
4.Calculate length of the string (use pointers)
5.Convert all lowercase characters to uppercase
6.Reverse the string
7.Insert a string in another string at a user specified position
8.Exit
Enter your choice:4
Length of Hello is:5
Length of WORLD is:5
```

Menu for string operation

```
1.Show address of each character in string
```

```
2.Concatenate two strings
3.Compare two strings
4.Calculate length of the string (use pointers)
5.Convert all lowercase characters to uppercase
6.Reverse the string
7.Insert a string in another string at a user specified position
8.Exit
Enter your choice:5
Hello after Uppercase HELLO
WORLD after Uppercase WORLD
```

Menu for string operation

```
1.Show address of each character in string
2.Concatenate two strings
3.Compare two strings
4.Calculate length of the string (use pointers)
5.Convert all lowercase characters to uppercase
6.Reverse the string
7.Insert a string in another string at a user specified position
8.Exit
Enter your choice:6
Hello after Reversing olleH
WORLD after Reversing dLROW
```

Menu for string operation

```
1.Show address of each character in string
2.Concatenate two strings
3.Compare two strings
4.Calculate length of the string (use pointers)
5.Convert all lowercase characters to uppercase
6.Reverse the string
7.Insert a string in another string at a user specified position
8.Exit
Enter your choice:7
```

```
Enter your choice:7
Enter the index:3
New string:HelWORLDlo
```

Menu for string operation

```
1.Show address of each character in string
2.Concatenate two strings
3.Compare two strings
4.Calculate length of the string (use pointers)
5.Convert all lowercase characters to uppercase
6.Reverse the string
7.Insert a string in another string at a user specified position
8.Exit
Enter your choice:8
..EXIT..
```

```
PS C:\Users\Niket\Documents\B.Sc. (H) C.S\SEM-2\C++> |
```

Q5- Write a program to merge two ordered arrays to get a single ordered array.

Program:

```
int main(){
    cout << "To merge two ordered Arrays" << endl;
    int n1, n2;
    do{
        cout << "Enter the no. of elements in array1 (must be integer
greater than 0):";
        cin >> n1;
    } while (n1 <= 0);
    int arr1[n1];
    inp(arr1, n1);
    cout << endl;
    do{
        cout << "Enter the no. of elements in array2 (must be integer
greater than 0):";
        cin >> n2;
    } while (n2 <= 0);
    int arr2[n2];
    inp(arr2, n2);
    cout << endl;
    cout << "Two ordered arrays are:" << endl;
    display(arr1, n1);
    display(arr2, n2);
    cout << endl;
    int merge[n1 + n2];
    int i = 0, j = 0, k = 0;
    while (i < n1 && j < n2){
        if (arr1[i] < arr2[j]){
            merge[k++] = arr1[i++];
        }else{
            merge[k++] = arr2[j++];
        }
    }
    while (i < n1){
        merge[k++] = arr1[i++];
    }
    while (j < n2){
        merge[k++] = arr2[j++];
    }
    cout << "Array after merge:" << endl;
    cout << "Array:";
    for (i = 0; i < n1 + n2; i++){
        cout << merge[i] << " ";
    }
    return 0;
}
```

Output:

```

To merge two ordered Arrays
Enter the no. of elements in array1 (must be integer greater than 0):3
Element 1:1
Element 2:2
Element 3:4

Enter the no. of elements in array2 (must be integer greater than 0):3
Element 1:3
Element 2:5
Element 3:6

Two ordered arrays are:
Array:1 2 4
Array:3 5 6

Array after merge:
Array:1 2 3 4 5 6
PS C:\Users\Niket\Documents\B.Sc. (H) C.S\SEM-2\C++>

```

Q6-Write a program to search a given element in a set of N numbers using Binary Search

- a. with recursion
- b. without recursion.

Program:

```
a) void Binarysearch(int arr[], int a, int l, int h){
    int mid = (l + h) / 2;
    if (l > h){
        cout << "Element not found";
        return;
    }
    if (arr[mid] == a){
        cout << "Element " << a << " found at index " << mid << endl;
        return;
    }else if (arr[mid] > a){
        h = mid - 1;
    }else{
        l = mid + 1;
    }
    Binarysearch(arr, a, l, h);
}

int main(){
    cout << "To find an element by Binary Search" << endl;
    int n, a;
    do{
        cout << "Enter the no. of elements in array(must be integer greater than 0
and sorted in ascending order):";
        cin >> n;
    } while (n <= 0);
    int arr[n];
    inp(arr, n);
    cout << endl;
    display(arr, n);
    cout << endl;
    cout << "Enter elements to search:";
    cin >> a;
    int l = 0;
    int h = n - 1;
    Binarysearch(arr, a, l, h);
    return 0;
}
```

Output:

```
To find an element by Binary Search
Enter the no. of elements in array(must be integer greater than 0 and sorted in ascending order):3
Element 1:1
Element 2:5
Element 3:7

Array:1 5 7

Enter elements to search:3
Element not found
PS C:\Users\Niket\Documents\B.Sc. (H) C.S\SEM-2\C++>
```

```

b) int main(){
    cout << "To find an element by Binary Search" << endl;
    int n, a;
    do{
        cout << "Enter the no. of elements in array(must be integer greater than 0
and sorted in ascending order):";
        cin >> n;
    } while (n <= 0);
    int arr[n];
    inp(arr, n);
    cout << endl;
    display(arr, n);
    cout << endl;
    cout << "Enter elements to search:";
    cin >> a;
    int l = 0;
    int h = n - 1;
    int mid = (l + h) / 2;
    while (l <= h){
        if (arr[mid] == a){
            cout << "Element " << a << " found at index " << mid << endl;
            break;
        }else if (arr[mid] > a){
            h = mid - 1;
        }else{
            l = mid + 1;
        }
        mid = (l + h) / 2;
    }
    if (l > h){
        cout << "Element not found";
    }
    return 0;
}

```

Output:

```

To find an element by Binary Search
Enter the no. of elements in array(must be integer greater than 0 and sorted in ascending order):4
Element 1:1
Element 2:3
Element 3:5
Element 4:7

Array:1 3 5 7

Enter elements to search:1
Element 1 found at index 0
PS C:\Users\Niket\Documents\B.Sc. (H) C.S\SEM-2\C++>

```

Q7-Write a program to calculate GCD of two numbers

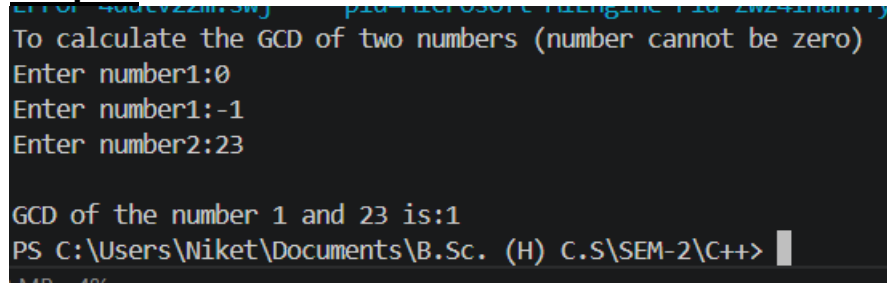
- a. with recursion
- b. without recursion.

Program:

```
a) void GCD(int a, int b, int i = 2, int j = 1){
    if (a == 1 && b == 1){
        cout << "GCD of the number " << a << " and " << b << " is:" << j;
        return;
    }
    if (a < 0)
        a = -a;
    if (b < 0)
        b = -b;
    if (i > a || i > b){
        cout << "GCD of the number " << a << " and " << b << " is:" << j;
        return;
    }
    if (a % i == 0 && b % i == 0){
        j = i;
    }
    GCD(a, b, i + 1, j);
}

int main(){
    cout << "To calculate the GCD of two numbers (number cannot be zero)" << endl;
    int a, b;
    do{
        cout << "Enter number1:";
        cin >> a;
    } while (a == 0);
    do{
        cout << "Enter number2:";
        cin >> b;
    } while (b == 0);
    cout << endl;
    GCD(a, b);
    return 0;
}
```

Output:



```
To calculate the GCD of two numbers (number cannot be zero)
Enter number1:0
Enter number1:-1
Enter number2:23

GCD of the number 1 and 23 is:1
PS C:\Users\Niket\Documents\B.Sc. (H) C.S\SEM-2\C++>
```

```

b) int main(){
    cout << "To calculate the GCD of two numbers (number cannot be zero)" << endl;
    int a, b;
    do{
        cout << "Enter number1:";
        cin >> a;
    } while (a == 0);
    do{
        cout << "Enter number2:";
        cin >> b;
    } while (b == 0);
    cout << endl;
    if (a < 0)
        a = -a;
    if (b < 0)
        b = -b;
    int i = 2, j = 1;
    while (i <= a && i <= b){
        if (a == 1 && b == 1){
            break;
        }
        if (a % i == 0 && b % i == 0){
            j = i;
        }
        i++;
    }
    cout << "GCD of the number " << a << " and " << b << " is:" << j;
    return 0;
}

```

Output:

```

To calculate the GCD of two numbers (number cannot be zero)
Enter number1:10
Enter number2:30

GCD of the number 10 and 30 is:10
PS C:\Users\Niket\Documents\B.Sc. (H) C.S\SEM-2\C++>

```

Q8- Create a Matrix class. Write a menu-driven program to perform following Matrix operations (exceptions should be thrown by the functions if matrices passed to them are incompatible and handled by the main() function):

- a. Sum
- b. Product
- c. Transpose

Program:

```
class Matrix {
    private:
        int row, col;
        int mat[10][10];
    public:
        Matrix(int r, int c, bool skip=false) {
            row = r;
            col = c;
            if (skip){
                return;
            }
            for(int i = 0; i < r; i++){
                for(int j = 0; j < c; j++){
                    cout << "Enter element (" << i+1 << ", " << j+1 <<
"): ";
                    cin >> mat[i][j];
                }
            }
            cout << "Matrix Created!!" << endl << endl;
        }
        void Sum(Matrix other){
            if (this->row == other.row && this->col == other.col){
                int r = other.row;
                int c = other.col;
                Matrix S(r, c, true);
                for(int i = 0; i < r; i++){
                    for(int j = 0; j < c; j++){
                        S.mat[i][j] = (this->mat[i][j] +
other.mat[i][j]);
                    }
                }
                S.Display();
            }else {
                throw "Not Compatible";
            }
        }
        void Product(Matrix other){
            if (this->col == other.row){
                int r1 = this->row;
                int k = other.row;
                int c2 = other.col;
                Matrix S(r1, c2,true);
```

```

        for(int i = 0; i < r1; i++){
            for(int j = 0; j < c2; j++){
                S.mat[i][j]=0;
                for(int K = 0; K < k; K++){
                    S.mat[i][j] += this->mat[i][K] *
other.mat[K][j];
                }
            }
        }
        S.Display();
    }else {
        throw "Not Compatible";
    }
}
void Transpose(){
    int r = this->row;
    int c = this->col;
    Matrix S(c, r, true);
    for(int i = 0; i < r; i++){
        for(int j = 0; j < c; j++){
            S.mat[j][i] = this->mat[i][j];
        }
    }
    S.Display();
}
void Display(){
    int r = this->row;
    int c = this->col;
    cout << "[ " << endl;
    for(int i = 0; i < r; i++){
        for(int j = 0; j < c; j++){
            cout << this->mat[i][j] << " ";
        }
        cout << endl;
    }
    cout << "]" << endl << endl;
}
};
int main(){
    cout << "Enter the Matrix for Matrix Operation" << endl << endl;
    int r,c;
    cout << "Enter Row for Matrix 1:";
    cin >> r;
    cout << "Enter Column for Matrix 1:";
    cin >> c;
    Matrix m1(r,c);
    cout << "Enter Row for Matrix 2:";
    cin >> r;
    cout << "Enter Column for Matrix 2:";
    cin >> c;
    Matrix m2(r,c);

```

```

cout << "m1:" << endl;
m1.Display();
cout << "m2:" << endl;
m2.Display();
while (true){
    cout << "\tMatrix Operations" << endl;
    cout << "1.Sum\n2.Product\n3.Transpose\n4.Exit";
    int x;
    cout << endl;
    cout << "Enter your choice:";
    cin >> x;
    switch (x){
        case 1:
            try{
                m1.Sum(m2);
            }catch(const char *msg){
                cout << msg << endl << endl;
            }
            break;
        case 2:
            try{
                m1.Product(m2);
            }catch(const char *msg){
                cout << msg << endl << endl;
            }
            break;
        case 3:
            cout << "Transpose of m1";
            m1.Transpose();
            cout << "Transpose of m2";
            m2.Transpose();
            break;
        case 4:
            cout << "..EXIT..";
            break;
        default:
            cout << "Enter Accordinly";
            break;
    }
    if (x==4){
        break;
    }
    cout << endl << endl;
}
return 0;
}

```

Output:

```
Enter the Matrix for Matrix Operation

Enter Row for Matrix 1:2
Enter Column for Matrix 1:3
Enter element (1,1): 1
Enter element (1,2): 2
Enter element (1,3): 3
Enter element (2,1): 4
Enter element (2,2): 5
Enter element (2,3): 6
Matrix Created!!

Enter Row for Matrix 2:3
Enter Column for Matrix 2:2
Enter element (1,1): 1
Enter element (1,2): 2
Enter element (2,1): 3
Enter element (2,2): 4
Enter element (3,1): 5
Enter element (3,2): 6
Matrix Created!!

m1:
[
1 2 3
4 5 6
]

m2:
[
1 2
3 4
5 6
]
```

```
Matrix Operations
1.Sum
2.Product
3.Transpose
4.Exit
Enter your choice:1
Not Compatible
```

```
Matrix Operations
1.Sum
2.Product
3.Transpose
4.Exit
Enter your choice:2
[
22 28
49 64
]
```

```
Matrix Operations
1.Sum
2.Product
3.Transpose
4.Exit
Enter your choice:3
Transpose of m1[
1 4
2 5
3 6
]
```

```
Transpose of m2[
1 3 5
2 4 6
]
```

```
Matrix Operations
1.Sum
2.Product
3.Transpose
4.Exit
Enter your choice:5
Enter Accordingly
```

```
Matrix Operations
1.Sum
2.Product
3.Transpose
4.Exit
Enter your choice:4
..EXIT..
```

Q9- Define a class Person having name as a data member. Inherit two classes Student and Employee from Person. Student has additional attributes as course, marks and year and Employee has department and salary. Write display() method in all the three classes to display the corresponding attributes. Provide the necessary methods to show runtime polymorphism.

Program:

```
class Person{
    protected:
        string Name;
    public:
        Person(string nam){
            Name=nam;
        }
        void Display(){
            cout << "Name: " << Name << endl;
            cout << "From Person Class..." << endl;
            return;
        }
};

class Student: public Person{
    private:
        string Course;
        float Marks;
        int Years;
    public:
        Student(string nam, string co, float m, int y):Person(nam){
            Course=co;
            Marks=m;
            Years=y;
        }
        void Display(){
            cout << "Name: " << Name << endl;
            cout << "Course: " << Course << endl;
            cout << "Marks: " << Marks << endl;
            cout << "Years: " << Years << endl;
            cout << "From Student Class..." << endl;
            return;
        }
};

class Employee: public Person{
    private:
        string Department;
        float Salary;
    public:
        Employee(string nam, string dep, float sal):Person(nam){
            Department=dep;
            Salary=sal;
        }
        void Display(){
```

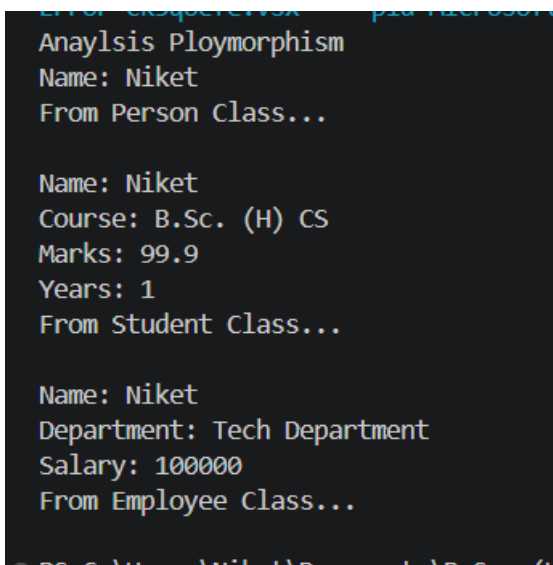
```

        cout << "Name: " << Name << endl;
        cout << "Department: " << Department << endl;
        cout << "Salary: " << Salary << endl;
        cout << "From Employee Class..." << endl;
        return;
    }
};

int main(){
    cout << "Anaylsis Ploymorphism" << endl;
    Person P1("Niket");
    P1.Display();
    cout << endl;
    Student S1("Niket","B.Sc. (H) CS",99.9,1);
    S1.Display();
    cout << endl;
    Employee E1("Niket","Tech Department",100000.00);
    E1.Display();
    cout << endl;
    return 0;
}

```

Output:



```

Anaylsis Ploymorphism
Name: Niket
From Person Class...

Name: Niket
Course: B.Sc. (H) CS
Marks: 99.9
Years: 1
From Student Class...

Name: Niket
Department: Tech Department
Salary: 100000
From Employee Class...

```

Q10- Create a Triangle class. Add exception handling statements to ensure the following conditions: all sides are greater than 0 and sum of any two sides is greater than the third side. The class should also have overloaded functions for calculating the area of a right angled triangle as well as using Heron's formula to calculate the area of any type of triangle.

Program:

```
class Triangle{
private:
    int a, b, c;
public:
    Triangle(int x, int y, int z){
        if (x <= 0 || y <= 0 || z <= 0){
            throw "Sides must be greater than 0";
        }
        if (!(x+y>z && y+z>x && z+x>y)){
            throw "Invalid triangle (triangle inequality failed)";
        }
        a=x;
        b=y;
        c=z;
    }
    void area(bool x){
        cout << "Area of right angles triangle with" << endl;
        cout << "Base:" << b << endl;
        cout << "Height:" << a << endl;
        cout << "Area:" << 0.5*a*b << endl;
        return;
    }
    void area(){
        double s=(a+b+c)/2.0;
        cout << "Area:" << sqrt(s*(s-a)*(s-b)*(s-c)) << endl;
    }
};

int main(){
    cout << "Enter the side of triangle to compute the area" << endl;
    int x, y, z;
    cout << "Enter side1:";
    cin >> x;
    cout << "Enter side2:";
    cin >> y;
    cout << "Enter side3:";
    cin >> z;
    try{
        char u;
        Triangle T(x, y, z);
        cout << "Is the triangle is Right angles?(T/any):";
        cin >> u;
        cout << endl;
        if(u=='T'){
```

```

        T.area(true);
    }else{
        T.area();
    }
}
catch (const char* msg){
    cout << "Exception: " << msg << endl;
}
return 0;
}

```

Output:

```

Enter the side of triangle to compute the area
Enter side1:0
Enter side2:0
Enter side3:0
Exception: Sides must be greater than 0
PS C:\Users\Niket\Documents\B.Sc. (H) C.S\SEM-2\
indowsDebugLauncher.exe' '--stdin=Microsoft-MIEn
Error-yzbkmlfb.hbr' '--pid=Microsoft-MIEngine-Pi
Enter the side of triangle to compute the area
Enter side1:1
Enter side2:2
Enter side3:3
Exception: Invalid triangle (triangle inequality
PS C:\Users\Niket\Documents\B.Sc. (H) C.S\SEM-2\
indowsDebugLauncher.exe' '--stdin=Microsoft-MIEn
Error-qwwoda4q.oby' '--pid=Microsoft-MIEngine-Pi
Enter the side of triangle to compute the area
Enter side1:3
Enter side2:4
Enter side3:5
Is the triangle is Right angles?(T/any):g

Area:6
PS C:\Users\Niket\Documents\B.Sc. (H) C.S\SEM-2\

```

```

Enter the side of triangle to compute the area
Enter side1:3
Enter side2:4
Enter side3:5
Is the triangle is Right angles?(T/any):T

Area of right angles triangle with
Base:4
Height:3
Area:6
PS C:\Users\Niket\Documents\B.Sc. (H) C.S\SEM-2\

```

Q11- Create a class Student containing fields for Roll No., Name, Class, Year and Total Marks. Write a program to store 5 objects of Student class in a file. Retrieve these records from the file and display them.

Program:

```
class Student{
public:
    int Rollno;
    string Name;
    int Class;
    float Total_Marks;
    int Years;
    Student(){}
    Student(int r,string nam, int c, float m, int y){
        Rollno=r;
        Name=nam;
        Class=c;
        Total_Marks=m;
        Years=y;
    }
    void Display(){
        cout << "Rollno: " << Rollno << endl;
        cout << "Name: " << Name << endl;
        cout << "Class: " << Class << endl;
        cout << "Total Marks: " << Total_Marks << endl;
        cout << "Years: " << Years << endl;
        return;
    }
};

int main(){
    Student S1;
    int r,c,y;
    string nam;
    float m;
    ofstream infile("C:\\Users\\Niket\\Documents\\B.Sc. (H) C.S\\SEM-2\\C++\\students.txt");
    cout << "Entering data of 5 students and storing in text file" << endl;
    for (int i=1; i<6; i++){
        cout << "Enter details of student " << i << endl;
        cout << "Rollno: ";
        cin >> r;
        cout << "Name: ";
        cin >> nam;
        cout << "Class: ";
        cin >> c;
        cout << "Total Marks: ";
        cin >> m;
        cout << "Years: ";
        cin >> y;
        Student S1(r,nam,c,m,y);
    }
}
```

```

        infile << S1.Rollno << " " << S1.Name << " " << S1.Class << " " << S1.Total_Marks << " "
<< S1.Years << "\n";
        cout << "Details of Student " << i << " is added" << endl << endl;
    }
    infile.close();
    cout << "Getting back Student Records from text file" << endl;
    ifstream outfile("C:\\Users\\Niket\\Documents\\B.Sc. (H) C.S\\SEM-2\\C++\\students.txt");
    int i=1;
    while (outfile >> S1.Rollno >> S1.Name >> S1.Class >> S1.Total_Marks >> S1.Years){
        cout << "Details of student " << i << endl;
        S1.Display();
        cout << endl;
        i++;
    }
    outfile.close();
    return 0;
}

```

Output:

```
Error-fo3kcu5t.mtm' '--pid=Microsoft-MIEngine-Pid-cu5e
Entering data of 5 students and storing in text file
Enter details of student 1
Rollno: 1
Name: abc
Class: 1
Total Marks: 100
Years: 1
Details of Student 1 is added

Enter details of student 2
Rollno: 2
Name: xyz
Class: 2
Total Marks: 100
Years: 2
Details of Student 2 is added

Enter details of student 3
Rollno: 3
Name: qwerty
Class: 3
Total Marks: 100
Years: 3
Details of Student 3 is added

Enter details of student 4
Rollno: 4
Name: pqr
Class: 4
Total Marks: 100
Years: 4
Details of Student 4 is added

Enter details of student 5
Rollno: 5
Name: mno
Class: 5
Total Marks: 100
Years: 5
Details of Student 5 is added
```

Getting back Student Records from text file

Details of student 1

Rollno: 1

Name: abc

Class: 1

Total Marks: 100

Years: 1

Details of student 2

Rollno: 2

Name: xyz

Class: 2

Total Marks: 100

Years: 2

Details of student 3

Rollno: 3

Name: qwerty

Class: 3

Total Marks: 100

Years: 3

Details of student 4

Rollno: 4

Name: pqr

Class: 4

Total Marks: 100

Years: 4

Details of student 5

Rollno: 5

Name: mno

Class: 5

Total Marks: 100

Years: 5

Practical File.cpp

students.txt X

students.txt

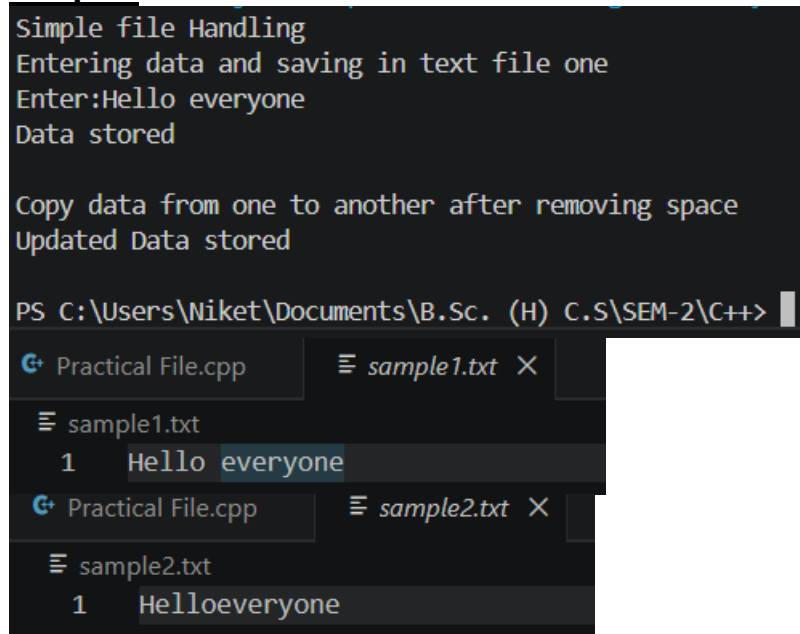
```
1 1 abc 1 100 1
2 2 xyz 2 100 2
3 3 qwerty 3 100 3
4 4 pqr 4 100 4
5 5 mno 5 100 5
6
```

Q12- Copy the contents of one text file to another file, after removing all whitespaces.

Program:

```
int main(){
    cout << "Simple file Handling" << endl;
    cout << "Entering data and saving in text file one" << endl;
    ofstream infile1("C:\\Users\\Niket\\Documents\\B.Sc. (H) C.S\\SEM-2\\C++\\sample1.txt");
    string data;
    cout << "Enter:";
    getline(cin, data);
    infile1 << data;
    cout << "Data stored" << endl << endl;
    infile1.close();
    cout << "Copy data from one to another after removing space" << endl;
    ifstream outfile1("C:\\Users\\Niket\\Documents\\B.Sc. (H) C.S\\SEM-2\\C++\\sample1.txt");
    ofstream infile2("C:\\Users\\Niket\\Documents\\B.Sc. (H) C.S\\SEM-2\\C++\\sample2.txt");
    string updated;
    getline(outfile1, data);
    for( char ch : data){
        if (ch != ' '){
            updated = updated + ch;
        }
    }
    infile2 << updated;
    cout << "Updated Data stored" << endl << endl;
    return 0;
}
```

Output:



The screenshot shows a Windows command prompt window with the following output:

```
Simple file Handling
Entering data and saving in text file one
Enter:Hello everyone
Data stored

Copy data from one to another after removing space
Updated Data stored
```

Below the command prompt, there are two file explorer windows. The first window shows the contents of 'sample1.txt' as 'Hello everyone'. The second window shows the contents of 'sample2.txt' as 'Helloeveryone', demonstrating that the spaces have been removed from the original text.